5. Pre-trained model. The pre-trained model can be trained on high-computing infrastructure like graphics processing unit (GPU) or on cloud infrastructure for several tasks like object detection, image classification, natural language processing, etc. The model can then be converted and deployed for mobile devices on different platforms.

Fig. 1 illustrates the workflow of TensorFlow Lite.

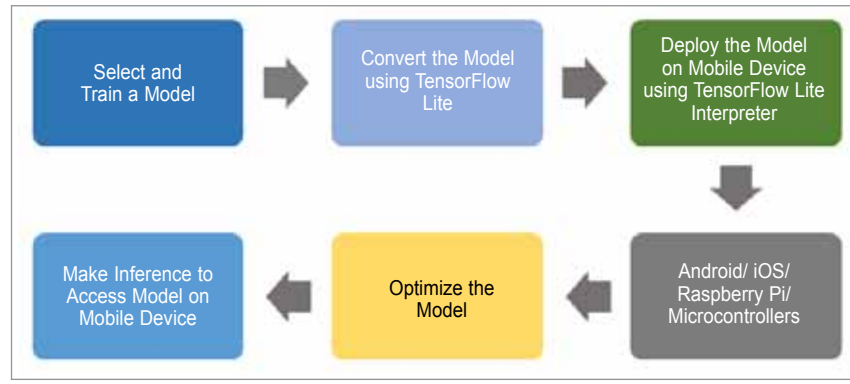

Fig. 1: Workflow of TensorFlow Lite

Model selection and training

Model selection and training is a crucial task for classification problems. There are several options available that can be used for this purpose.

- Build and train a custom model using application-specific dataset
- Apply a pre-trained model like ResNet, MobileNet, InceptionNet, or NASNetLarge, which are already trained on ImageNet dataset
- Train the model using transfer learning for the desired classification problem

Model conversion using TensorFlow Lite

Trained models can be converted to the TensorFlow Lite version. TensorFlow Lite is a special model format which is light-weight, accurate, and suitable for mobile and embedded devices. Fig. 2 shows the TensorFlow Lite conversion process.

Using TensorFlow converter, the TensorFlow model is converted to a flat buffer file called '.tflite' file. This .tflite file is what is deployed to the mobile or embedded device.

The trained model obtained after the training process is required to be saved. The saved model is an architecture that stores different information related to weights, biases, and training configuration in a single file. Thus, sharing and deploying of the model becomes easy with the saved model.

The following code is used to convert the saved model to TensorFlow Lite.

```
#save the model after compiling
model.save('test_keras_model.h5')
keras_model=tf.keras.models.load_model
(' test_keras_model.h5')

#conver the tf.keras model to TensorFlow
Lite model
converter=tf.lite.TFLiteConverter.from_
keras_model(keras_model)
tflite_model=converter.convert()
```

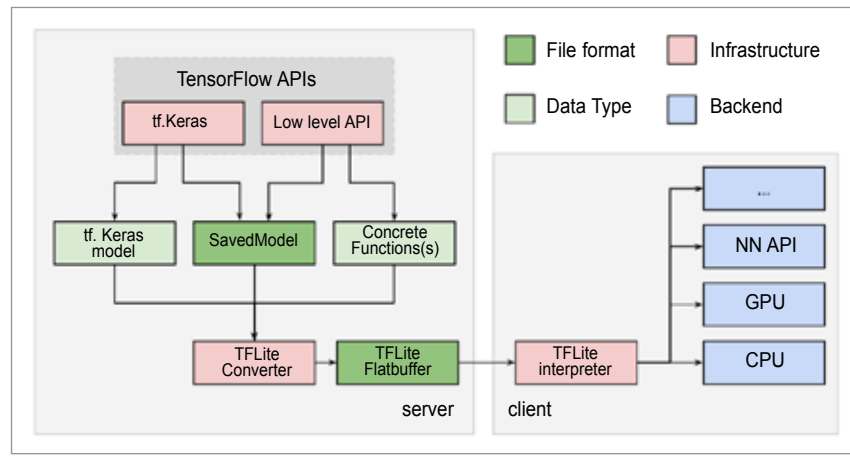

Fig. 2: Conversion process of TensorFlow Lite (Image source: www.tensorflow.org/lite/convert/index)

Model optimisation

An optimised model demands less space and resources on hand-held devices like mobile and embedded devices. Therefore, TensorFlow Lite uses two special approaches—Quantisation and Weight Pruning—to achieve model optimisation.

Quantisation makes the model light-weight; it refers to the process of reducing the precision of the numbers which are used to represent the different parameters of the TensorFlow model. Weight pruning is the process of trimming the parameters in the model which have less impact on the overall model performance.

Make inference to access models on mobile devices

The TensorFlow Lite model can be deployed on many types of hand-held devices like Raspberry Pi, Android, iOS, or microcontrollers. Following the below steps can ensure a mobile interface that allows you to access the ML/DL model easily and accurately.

- With the model, initialise and load interpreter
- Assign the tensors and obtain input and output tensors
- Read and pre-process the image by loading it into a tensor
- Use and invoke an interpreter that makes the inference on the input tensor
- Generate the result of the image by mapping the result from the inference EFY

This article was first published in March 2021 issue of Open Source For You